

TINKERING LAB MANUAL

Course Code: 23AC003

Department of Computer Science And Engineering



**SRI VENKATESWARA COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)**

Accredited by NBA, New Delhi s NAAC, Bengaluru

Affiliated to JNTUA, Ananthapuram, Recognized by the UGC under

Section 12(B) and 12(F) | Approved by AICTE, New Delhi.

R.V.S. NAGAR, TIRUPATI ROAD, CHITTOOR – 517127 (A.P) – INDIA

Prepared By:

Dr George Justy Mirobi

Department of CSE
SVCET, Chittoor

(23ACO03) TINKERING LAB

The aim of tinkering lab for engineering students is to provide a hands-on learning environment where students can explore, experiment, and innovate by building and testing prototypes. These labs are designed to demonstrate practical skills that complement theoretical knowledge.

Course objectives: The objectives of the course are to	
1	Encourage Innovation and Creativity
2	Provide Hands-on Learning and Impart Skill Development
3	Foster Collaboration and Teamwork
4	Enable Interdisciplinary Learning, Prepare for Industry and Entrepreneurship
5	Impart Problem-Solving mind-set

These labs bridge the gap between academia and industry, providing students with the practical experience. Some students may also develop entrepreneurial skills, potentially leading to start-ups or innovation-driven careers. Tinkering labs aim to cultivate the next generation of engineers by giving them the tools, space, and mind-set to experiment, innovate, and solve real-world challenges.

List of experiments:

- 1) Make your own parallel and series circuits using breadboard for any application of your choice.
- 2) Demonstrate a traffic light circuit using breadboard.
- 3) Build and demonstrate automatic Street Light using LDR.
- 4) Simulate the Arduino LED blinking activity in Tinkercad.
- 5) Build and demonstrate an Arduino LED blinking activity using Arduino IDE.
- 6) Interfacing IR Sensor and Servo Motor with Arduino.
- 7) Blink LED using ESP32.
- 8) LDR Interfacing with ESP32.
- 9) Control an LED using Mobile App.
- 10) Design and 3D print a Walking Robot
- 11) Design and 3D Print a Rocket.
- 12) Build a live soil moisture monitoring project, and monitor soil moisture levels of a remote plan in your computer dashboard.
- 13) Demonstrate all the steps in design thinking to redesign a motor bike.

Students need to refer to the following links:**Course Outcomes:** The students will be able to experiment, innovate, and solve real-world challenges.

- 1) <https://aim.gov.in/pdf/equipment-manual-pdf.pdf>
- 2) <https://atl.aim.gov.in/ATL-Equipment-Manual/>
- 3) <https://aim.gov.in/pdf/Level-1.pdf>
- 4) <https://aim.gov.in/pdf/Level-2.pdf>
- 5) <https://aim.gov.in/pdf/Level-3.pdf>

GENERAL INTRODUCTION TO ELECTRONICS AND CIRCUITS

Electronics is all around us like from the toys we play with to the gadgets we use every day. If you've ever opened up a battery-operated toy or a household device, you've probably noticed a battery, some wires, and maybe a small switch. These simple components work together to form an electronic circuit, which is a closed loop that allows electric current to flow.

A circuit can be thought of as a circular path that connects a power source (like a battery) to different components (like LEDs, resistors, or buzzers) using conductive wires. The switch controls whether the current can flow, just like a gate.

Learning about basic electronics helps us understand how electrical devices function. By exploring simple circuits, we also build the foundation to create more advanced projects in the future. In this module or activity, you will get hands-on experience with real components such as breadboards, batteries, LEDs, and resistors.

Understanding Closed and Open Circuits

A closed circuit works like a complete circle — it starts and ends at the same point, forming a continuous loop. This allows electricity to flow smoothly from the positive terminal of the power source to the negative terminal. In a closed circuit, all components (like a bulb or an LED) receive power and work as intended.

In contrast, an open circuit happens when there's a break anywhere along the path. This break stops the flow of electric current, which means devices like bulbs won't light up. A simple switch can open or close a circuit by connecting or disconnecting the pathway.

Key point:

- **Closed Circuit:** Current flows — the device works.
- **Open Circuit:** Current is blocked — the device does not work.



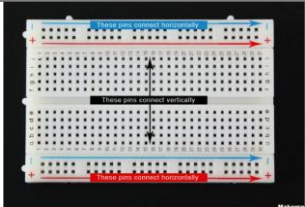
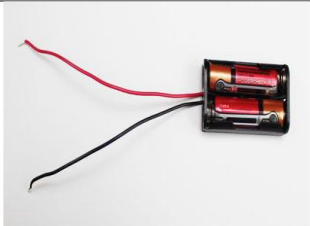
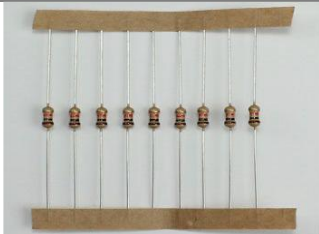
Example:

In the picture:

- The left diagram shows a bulb glowing in a closed circuit.
- The right diagram shows the same bulb off because the switch is open, breaking the circuit.

ELECTRICAL TOOLS

Working with electricity can be dangerous if not done with proper tools. These tools are essential to safely build electronic devices. Let's take a look at a few of them:

S.No	Tool	Description	Image
1	Breadboard	An essential tool for building circuits. These boards contain holes for inserting wires and components without soldering.	
2	Digital Multimeter	A device that measures electric current (amps), voltage (volts), and resistance (ohms).	
3	Battery Holder	A plastic case that holds batteries from 9V to AA, providing portable power.	
4	Wire Cutter / Stripper	Wire cutters are essential for stripping insulation and cutting copper wires to size.	
5	Light-Emitting Diode (LED)	A light source that glows when an electric current flows in one direction. LEDs are used in indicators, displays, and more.	
6	Resistors	Resistors limit the flow of electric current in a circuit. They help protect components by controlling voltage and current.	

7	Heat Gun	Used to shrink plastic tubing known as heat shrink, which keeps wires neat and safe.	
8	Soldering Iron	A tool to join electronic parts by melting solder to hold them together permanently.	

STUDY EXERCISE: GETTING STARTED WITH TINKERCAD FOR CIRCUIT DESIGN

Objective

To learn the basics of using [Tinkercad](https://www.tinkercad.com) for building and simulating simple electronic circuits, including using LEDs, resistors, and push buttons on a breadboard.

Prerequisites

- Basic knowledge of electronic components (LEDs, resistors, breadboard, battery)
- An internet-connected device with a web browser

Exercise Outline

Part 1: Creating a Tinkercad Account

1. Go to <https://www.tinkercad.com>
2. Click **Join Now** → **Create a personal account** or **Log In** if you already have an account.
3. Use your email ID or Google account to sign up.

Part 2: Starting a New Circuit

1. After logging in, click on the "**Circuits**" tab from the left panel.
2. Click "**Create New Circuit**".
3. You will be taken to the circuit design workspace.

Part 3: Adding Components

1. In the components panel, **drag and drop** the following:
 - 1 Breadboard (small)
 - 1 LED (any color)
 - 1 Resistor (330Ω or 1kΩ)
 - 1 Push Button
 - 1 9V Battery or Power Supply
 - Wires (click on a pin and drag to another point to connect)

Part 4: Making Connections

1. Connect the **positive terminal of the battery** to the breadboard's **power rail (+)**.
2. Connect the **negative terminal** to the **ground rail (-)**.
3. Place the **LED** on the breadboard.
 - Connect a **resistor to the anode (long leg)** of the LED and then to the positive rail.
 - Connect the **cathode (short leg)** of the LED to one terminal of the push button.
 - Connect the other terminal of the push button to the ground rail.

Part 5: Simulating the Circuit

1. Click the **Start Simulation** button (top right).
2. Press the push button in the simulation.
3. Observe the LED glowing.
4. Try different resistor values or add more LEDs to experiment.

Tasks for Practice

Task No.	Task Description
1	Build a traffic light system with 3 LEDs and push buttons.
2	Add a buzzer that sounds when any LED is turned on.
3	Use a switch to turn the entire circuit ON or OFF.
4	Add a delay using an Arduino code to blink LEDs sequentially (optional advanced).

STUDY EXERCISE: GETTING STARTED WITH ARDUINO BOARD

Objective

To learn the basics of working with an Arduino board, understand its features and pin configuration, connect simple input and output components, upload basic code, and observe the results.

Prerequisites

Basic understanding of electronic components like LEDs, resistors, push buttons, and breadboards. Familiarity with connecting circuits and using a computer to upload code. An internet connection is required for Tinkercad or the Arduino IDE.

Arduino Board Features

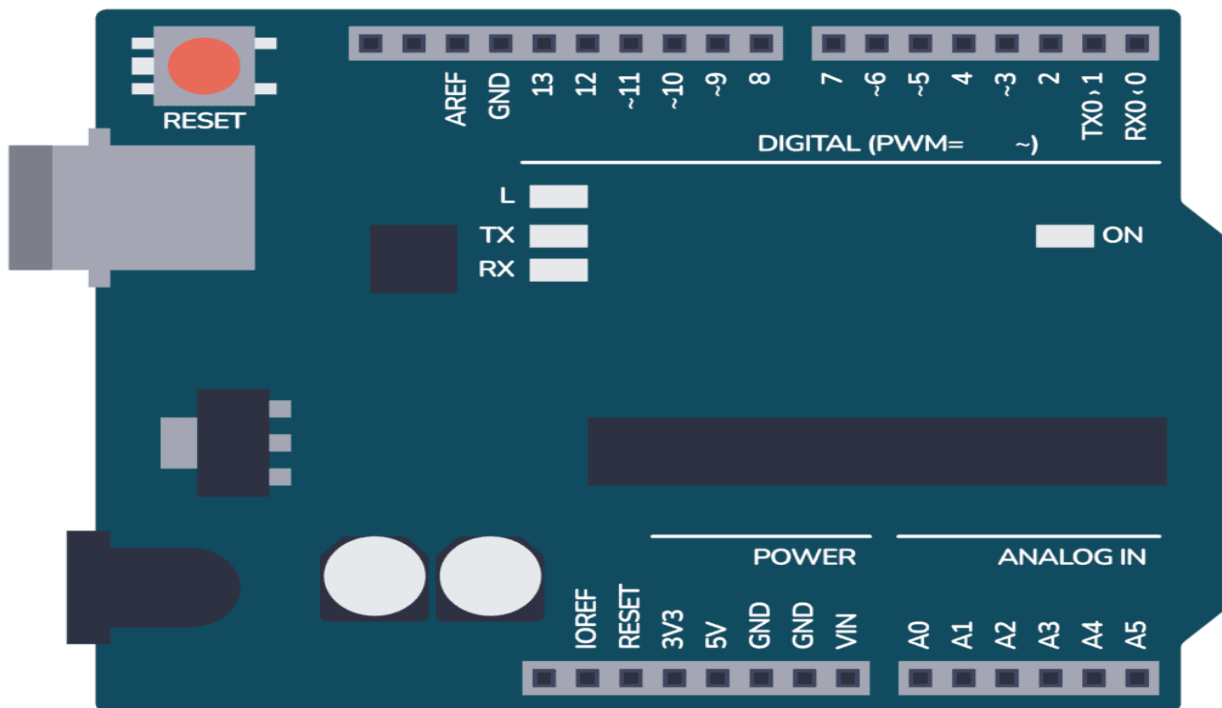
The Arduino Uno is one of the most commonly used microcontroller boards for beginners. It is based on the ATmega328P microcontroller and features the following:

- 14 digital input/output (I/O) pins (of which 6 can provide PWM output)
- 6 analog input pins (A0 to A5) for reading sensor values
- A USB port for programming and power supply
- A power jack for external power (7–12V DC recommended)
- A reset button to restart the program
- An onboard LED connected to digital pin 13
- Voltage regulator and crystal oscillator for stable operation

Pin Configuration

- Digital Pins (0–13): Used for digital input or output. Pins 3, 5, 6, 9, 10, and 11 support PWM (Pulse Width Modulation).
- Analog Pins (A0–A5): Used for reading analog sensor values (0–1023).
- Power Pins:
 - Vin: Input voltage to the Arduino when using an external power source.
 - 5V: Regulated 5V output.
 - 3.3V: 3.3V output.
 - GND: Ground pins (multiple).
- AREF: Reference voltage for analog inputs.
- Reset: Can be used to reset the microcontroller.

Board Drawing (Description)



A typical Arduino Uno board diagram includes:

- The USB port on one end for programming.
- A power jack for connecting an external DC adapter.
- The reset button near the USB port.
- The row of digital pins (0–13) on one side.
- The row of analog pins (A0–A5) on the other side.
- A small onboard LED (labelled L) near pin 13.
- The microcontroller chip in the center of the board.
- The power pins (GND, 5V, 3.3V, Vin) grouped together near the analog pins.

Working Procedure

Connect the Arduino Uno board to your computer using a USB cable. Install the Arduino IDE from www.arduino.cc if you are using a physical board. Alternatively, open [Tinkercad Circuits](#) to simulate Arduino projects online.

Open a new sketch in the Arduino IDE or the Tinkercad code editor. Place an LED and a resistor on a breadboard. Connect the anode (long leg) of the LED to a digital output pin such as pin 13. Connect the cathode (short leg) through a resistor to a GND pin on the Arduino.

Write a simple program to blink the LED. Use the `setup()` function to set the pin mode and the `loop()` function to turn the LED on and off with delays. Select the correct board (Arduino Uno) and port from the Tools menu in the IDE. Click Upload to transfer the code to the board. If using Tinkercad, click Start Simulation to run the program.

Observe the LED blinking on and off. Change the delay values to see how the blink rate changes. Add push buttons or additional LEDs to explore more inputs and outputs. Students can also run

the built-in Blink example found in the IDE under File → Examples → 01.Basics → Blink to test the onboard LED.

Result

By the end of this exercise, students will be able to identify the features and pin configuration of an Arduino Uno board, connect basic electronic components to the board, write and upload simple programs, and observe how the microcontroller controls outputs based on the code.

Ex. No 1: MAKE YOUR OWN PARALLEL AND SERIES CIRCUITS USING A BREADBOARD — *Application: LED Lighting*

Objective:

To design, build, and compare the behavior of LEDs connected in **series** and **parallel** using a breadboard.

Components Required:

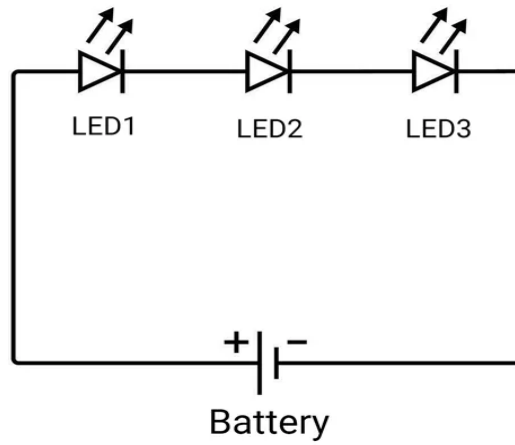
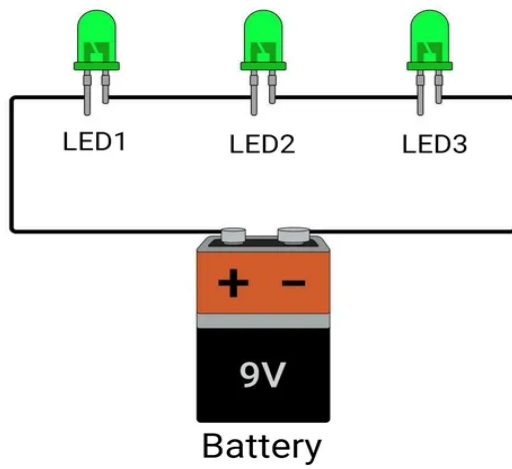
S. No	Component	Specification	Quantity
1	Breadboard	Standard	1
2	LEDs	5mm Red LEDs	3
3	Resistors	330 Ω (for series) 470–1k Ω (for parallel)	3
4	Connecting Wires	Jumper Wires	As needed
5	Power Source	9V Battery with Snap Connector	1

Working Theory

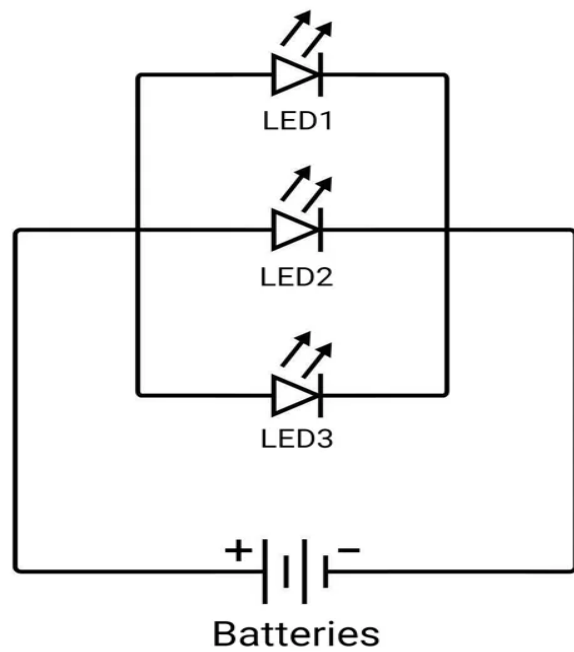
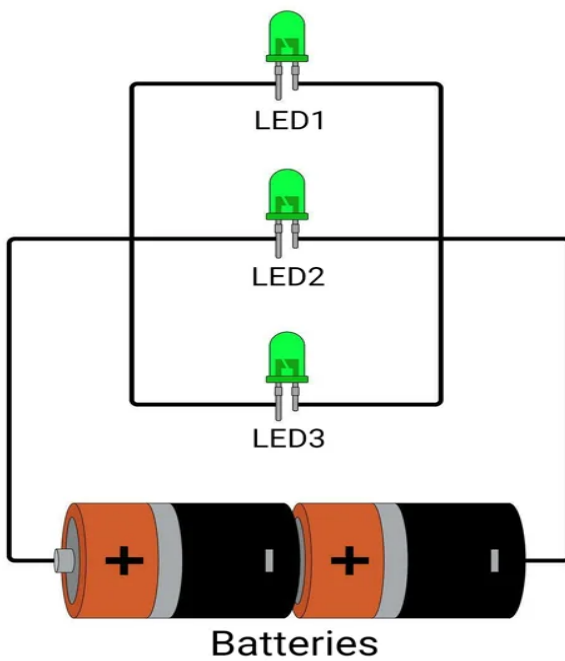
Aspect	Series Circuit	Parallel Circuit
Connection	LEDs are connected end-to-end; anode of one LED to cathode of the next.	Each LED is connected in a separate branch; all anodes joined together and all cathodes joined together.
Voltage Distribution	Supply voltage is divided among the three LEDs. For example, 5 V is shared (~ 1.6 – 1.7 V per LED).	Each LED branch receives the full supply voltage (e.g., 5 V).
Current Flow	Same current flows through all LEDs.	Total current is divided among the branches; each LED branch draws its own current independently.
Effect of Removing an LED	Removing or failing one LED breaks the circuit; all LEDs turn off.	Other LEDs continue to glow even if one LED is removed or fails.
Brightness	LEDs glow dimmer due to voltage sharing.	LEDs glow brighter and uniformly because each gets the full supply voltage.
Reliability	Less reliable — failure in one LED affects all.	More reliable — each LED operates independently; failure does not affect the others.
Application Suitability	Less suitable for applications where consistent brightness and reliability are needed.	Suitable for decorative lights, indicator panels, and applications needing consistent brightness and reliability.

Circuit Diagram:

Series Circuit

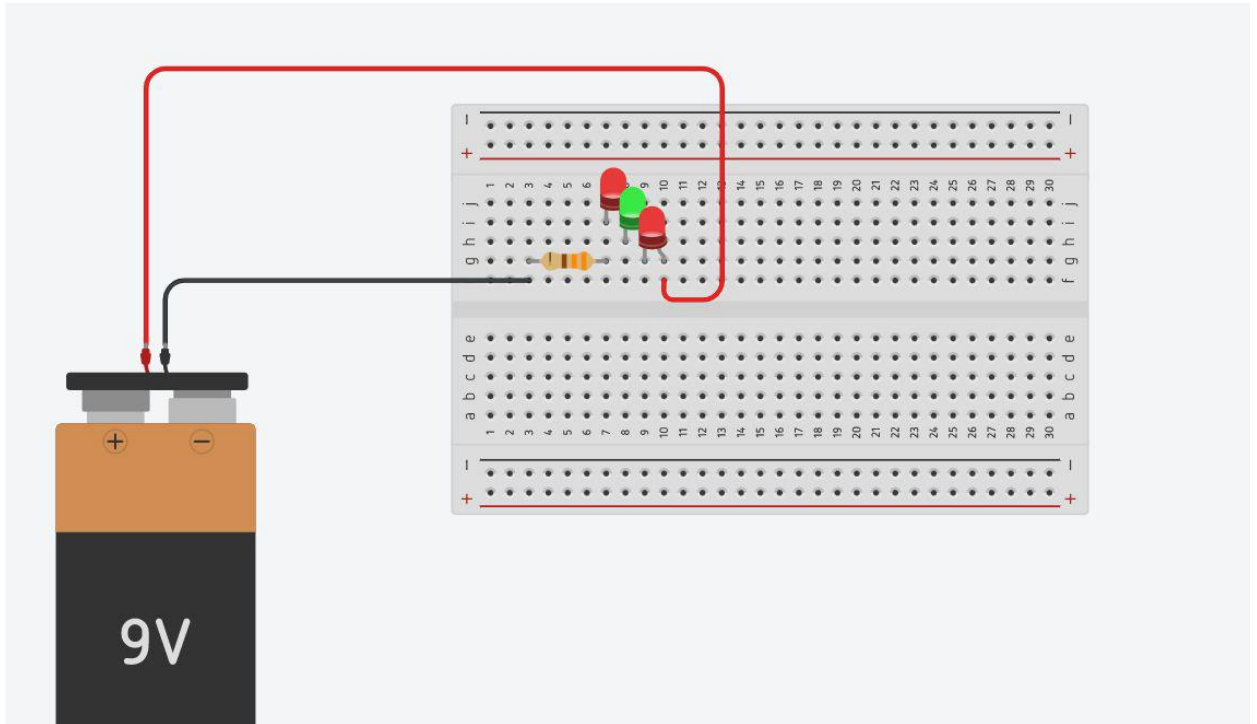


Parallel Circuit



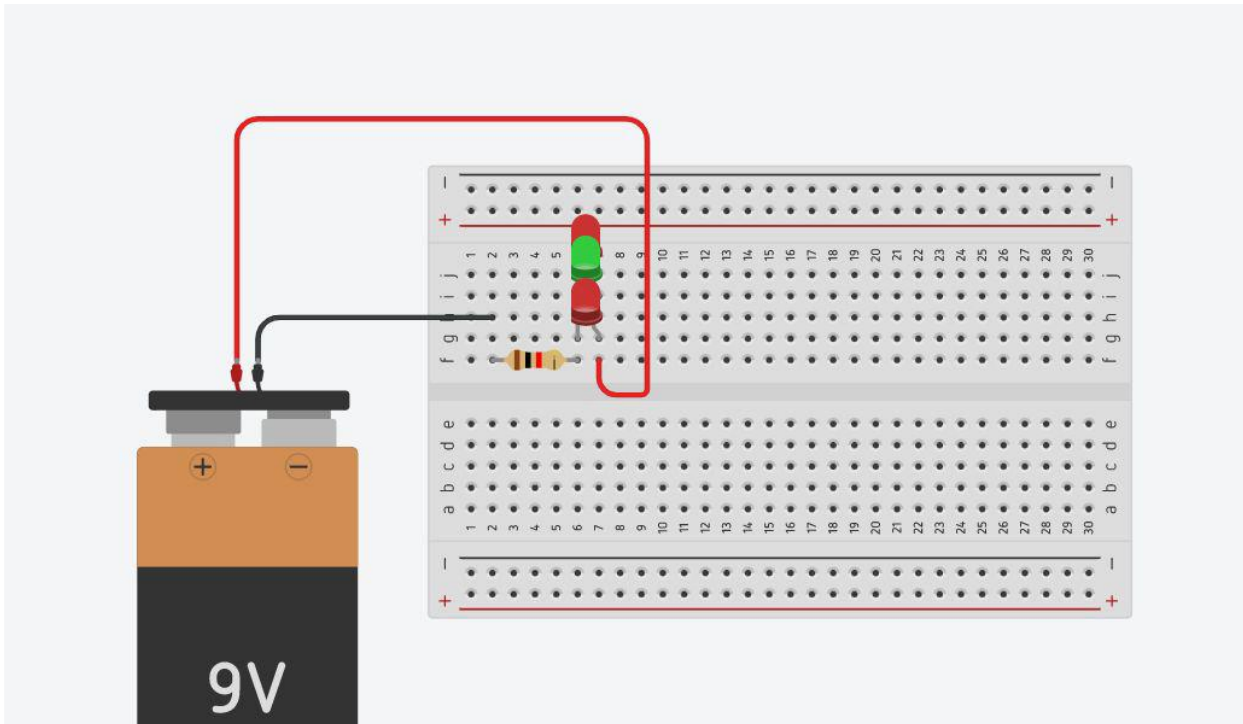
Procedure:

A) Series Circuit



1. Place 3 LEDs on the breadboard in line.
2. Connect the anode (+) of the first LED to the cathode (–) of the second LED; connect the anode of the second LED to the cathode of the third LED.
3. Connect a resistor (220 Ω) in series — either before the first LED or after the third LED.
4. Connect the free anode to the positive terminal of the power supply and the free cathode to the negative terminal.

B) Parallel Circuit



1. Insert 3 LEDs on separate rows.
2. Connect all anodes together (common positive) and all cathodes together (common negative).
3. Use individual resistors ($220\ \Omega$) for each LED branch to limit current.
4. Connect the combined anode line to the positive terminal and the combined cathode line to the negative terminal of the power supply.

Observation Table

Type of Circuit	No. of LEDs Connected	No. of LEDs Glowing	Current Through Each LED	Observation When One LED is Removed
Series Circuit	3	3 (all glow dimmer)	Same current for all ($\sim 20\text{ mA}$)	All LEDs turn off — entire path is broken.
Parallel Circuit	3	3 (all glow brightly)	Each LED draws own ($\sim 20\text{ mA}$)	Remaining LEDs continue to glow — unaffected.

Result:

Using a 9V battery, the parallel circuit is safer and more practical for LED lighting: consistent brightness, reliable operation, and failure of one LED does not affect the others. Proper resistor sizing is crucial to avoid burning out LEDs.

Note: Submit your Tinkercad link and your completed observation table for review.

EX NO 2 — TRAFFIC LIGHT CIRCUIT USING BREADBOARD

Aim

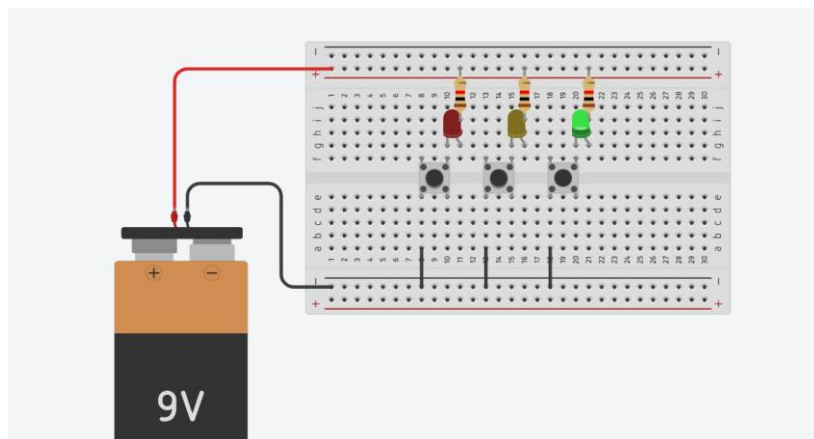
To design and assemble a simple traffic light circuit using a breadboard, LEDs, resistors, push buttons, and a 9V battery, and to observe its working.

S. No	Component	Specification	Quantity
1	Breadboard	Standard size	1
2	LEDs	Red, Yellow, Green	1 each
3	Resistors	330 Ω – 1k Ω	3
4	Push Buttons	SPST push buttons	3
5	Connecting Wires	Male-to-male	As required
6	Battery	9V with battery clip	1

Working Theory

A traffic light circuit simulates how traffic signals operate. Each LED represents a traffic signal (Red = Stop, Yellow = Wait, Green = Go). Each LED is connected in series with a resistor to limit current, preventing LED damage. Push buttons act as manual switches: when pressed, they complete the circuit for that LED, allowing it to light up. Releasing the button breaks the circuit, turning the LED off. This setup demonstrates basic series circuit connections and manual control.

Circuit Diagram Description



- The 9V battery powers the circuit.
- The positive terminal connects to the breadboard's positive rail; the negative terminal connects to the ground rail.
- Each LED is connected in series with a resistor and a push button.
- The other end of each push button is connected to the ground rail.
- When a push button is pressed, current flows through the LED–resistor branch, lighting up the LED.

Procedure

1. Connect the 9V battery to the breadboard's power rails (red wire to +, black wire to –).
2. Place the Red, Yellow, and Green LEDs on the breadboard.
3. Connect a **resistor** in series with each LED's anode (longer leg).
4. Connect each resistor to the positive rail.
5. Connect the cathode (shorter leg) of each LED to one terminal of a push button.
6. Connect the other terminal of each push button to the ground rail.
7. Verify all connections are secure.
8. Press each push button to observe the respective LED glow.

S. No	LED Color	Push Button Status	LED Status	Remarks
1	Red	Pressed	Red LED glows	Working correctly
2	Yellow	Pressed	Yellow LED glows	Working correctly
3	Green	Pressed	Green LED glows	Working correctly
4	Any LED	Button Released	LED turns off	Working correctly

Result

The traffic light circuit was successfully implemented on a breadboard. The LEDs glowed as expected when the respective push buttons were pressed, demonstrating the working of a simple traffic light system.

Note: Submit your Tinkercad link and your completed observation table for review.

EX. No 3: TRAFFIC LIGHT CIRCUIT USING ARDUINO IN TINKERCAD SIMULATION

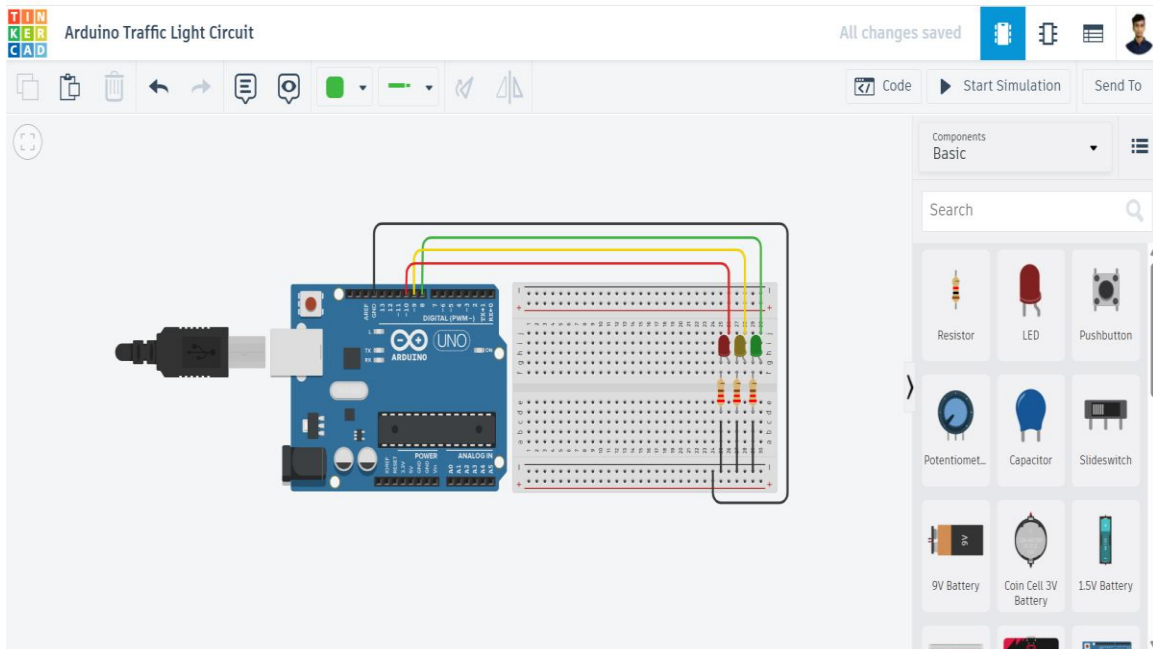
Objective:

To simulate a traffic light system using Arduino Uno, LEDs, and a breadboard. This project demonstrates the sequential control of traffic signals: Green → Yellow → Red, including a blinking green feature.

Components Required:

S.No	Component	Quantity
1	Arduino Uno R3	1
2	Breadboard	1
3	Green LED	1
4	Yellow LED	1
5	Red LED	1
6	Resistors (220Ω)	3
7	Jumper Wires	As required
8	USB Cable	1

Circuit Connections:



Procedure

- Open your Tinkercad Traffic Light Circuit.
- Connect Red, Yellow, and Green LEDs to Arduino pins 10, 9, and 8 respectively through 220Ω resistors.
- Connect the cathodes of all LEDs to GND.
- Paste the code in the Code Editor.
- Start the Simulation.
- Observe the following behavior: Green blinks 10 times → stays ON for 15s, Yellow ON for 5s, Red ON for 15s.

LED Color	Arduino Pin	Breadboard Connection
Red	Pin 10	220Ω resistor to anode; cathode to GND
Yellow	Pin 9	220Ω resistor to anode; cathode to GND
Green	Pin 8	220Ω resistor to anode; cathode to GND

Arduino Code:

```

void setup() {
  Serial.begin(9600);          // Start serial communication
  pinMode(10, OUTPUT); // Red LED
  pinMode(9, OUTPUT);  // Yellow LED
  pinMode(8, OUTPUT);  // Green LED
  Serial.println("Traffic light simulation starting...");
}

void loop() {
  // Blink green LED 10 times
  Serial.println("Blinking Green LED 10 times...");
  for (int i = 0; i < 10; i++) {
    digitalWrite(8, HIGH);
    Serial.println("Green LED ON");
    delay(100);
    digitalWrite(8, LOW);
    Serial.println("Green LED OFF");
    delay(100);
  }

  // Green ON for 15 seconds
  digitalWrite(8, HIGH);

```

```

Serial.println("Green LED ON for 15 seconds...");
delay(15000);
digitalWrite(8, LOW);
Serial.println("Green LED OFF");

// Yellow ON for 5 seconds
digitalWrite(9, HIGH);
Serial.println("Yellow LED ON for 5 seconds...");
delay(5000);
digitalWrite(9, LOW);
Serial.println("Yellow LED OFF");

// Red ON for 15 seconds
digitalWrite(10, HIGH);
Serial.println("Red LED ON for 15 seconds...");
delay(15000);
digitalWrite(10, LOW);
Serial.println("Red LED OFF");

Serial.println("Restarting sequence...\n");
}

```

Output Observation:

Step	LED	State	Duration
1	Green	Blinking	1 sec (10x100ms)
2	Green	ON	15 seconds
3	Yellow	ON	5 seconds
4	Red	ON	15 seconds

Result:

The Arduino-based traffic light circuit was successfully simulated using Tinkercad. LEDs operated in the standard traffic light sequence with an advanced blinking green signal.

Note: Submit your Tinkercad link and your completed observation table for review.

Date: _____

EX. NO 4: AUTOMATIC STREET LIGHTING USING LDR SENSOR

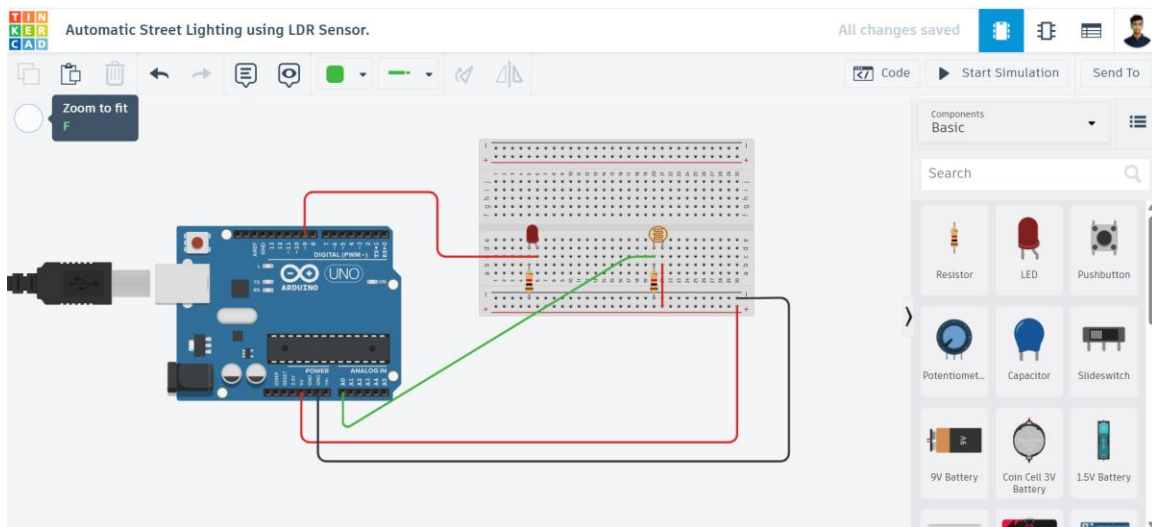
Objective:

To design and simulate an automatic street light system using an LDR (Light Dependent Resistor) and Arduino Uno. The system should automatically turn ON the LED (street light) during darkness and turn it OFF during daylight.

Components Required:

S.No	Component	Quantity
1	Arduino Uno R3	1
2	Breadboard	1
3	LDR Sensor	1
4	LED (White/Yellow)	1
5	10k Ω Resistor	1
6	220 Ω Resistor	1
7	Jumper Wires	As required
8	USB Cable	1

Circuit Connections:



1. Connect one terminal of the LDR to 5V.
2. Connect the other terminal of the LDR to A0 and to one end of a 10k Ω resistor.
3. Connect the other end of the resistor to GND (this forms a voltage divider).
4. Connect the anode of the LED to Digital Pin 9 via a 220 Ω resistor.
5. Connect the cathode of the LED to GND.

Arduino Code:

```

int ldrPin = A0;
int ledPin = 9;
int ldrValue = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  ldrValue = analogRead(ldrPin);
  Serial.println(ldrValue);

  if (ldrValue < 500) { // darkness
    digitalWrite(ledPin, HIGH); // Turn ON street light
  } else {
    digitalWrite(ledPin, LOW); // Turn OFF street light
  }
  delay(500);
}

```

Procedure:

- Open your Tinkercad simulation for Automatic Street Lighting.
- Connect the LDR in a voltage divider configuration with a 10k Ω resistor.
- Connect the LED to Digital Pin 9 via a 220 Ω resistor.
- Upload the given code in the Tinkercad code editor.
- Start the simulation and vary the light on the LDR using the simulation tool.
- Observe that the LED turns ON in darkness and OFF in light.

Output Observation:

Condition	LDR Reading (approx)	LED Status
Bright Light	> 500	OFF
Dim/Dark	< 500	ON

Result:

The automatic street lighting system was successfully simulated. The LED turns ON in darkness and OFF in brightness based on the LDR input.

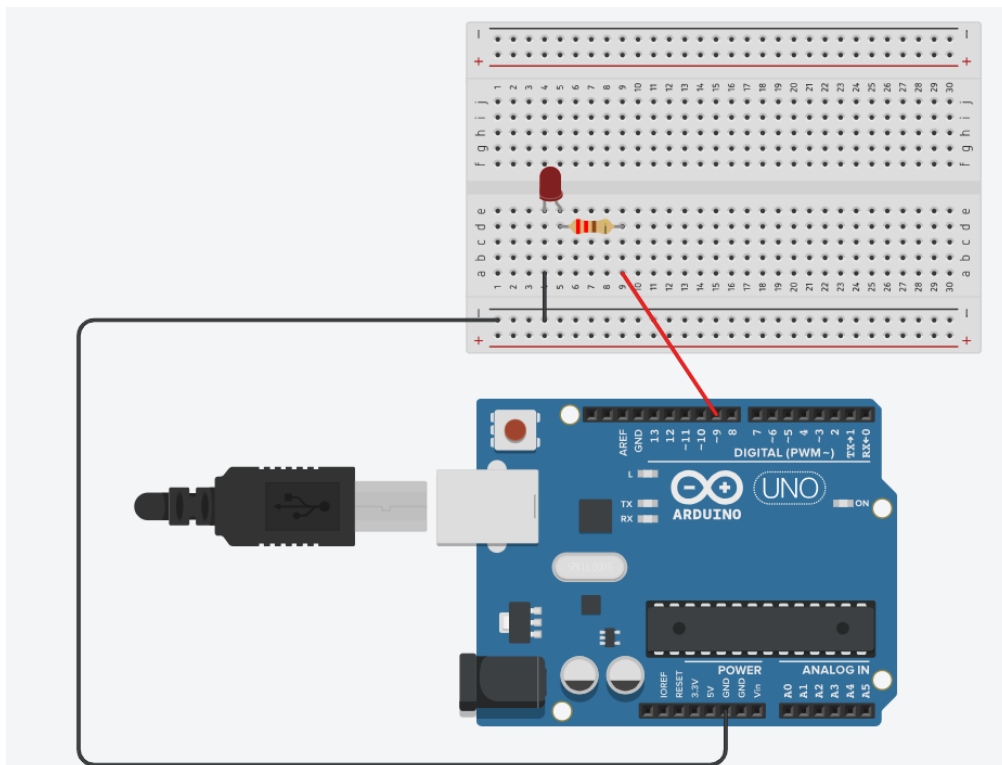
Note: Submit your Tinkercad link and your completed observation table for review.

EX. NO 5: LED BLINKING USING ARDUINO IN TINKERCAD SIMULATION**Objective:**

To simulate a basic LED blinking program using Arduino Uno and understand the use of digital output pins.

Components Required:

S.No	Component	Quantity
1	Arduino Uno R3	1
2	Breadboard	1
3	LED (Any Color)	1
4	220Ω Resistor	1
5	Jumper Wires	As required
6	USB Cable	1

Circuit Connections:

Circuit Connection Table:

Component	Connection to Arduino	Additional Notes
LED (Anode)	Digital Pin 9 via 220Ω resistor	Positive terminal of LED
LED (Cathode)	GND	Negative terminal of LED

Arduino Code:

```
void setup() {
  pinMode(9, OUTPUT); // Set pin 9 as output
}

void loop() {
  digitalWrite(9, HIGH); // Turn LED on
  delay(1000);           // Wait for 1 second
  digitalWrite(9, LOW);  // Turn LED off
  delay(1000);           // Wait for 1 second
}
```

Procedure:

- Open the Tinkercad circuit for LED Blinking.
- Connect the LED anode to Digital Pin 9 via a 220Ω resistor.
- Connect the LED cathode to GND.
- Paste the code into the code editor in Tinkercad.
- Start the simulation and observe the LED blinking at 1-second intervals.
- Output Observation:
- The LED turns ON for 1 second and then OFF for 1 second repeatedly.

Result:

The LED blinking simulation was successfully performed in Tinkercad using Arduino Uno.

Ex. 6: Arduino LED Blinking Activity

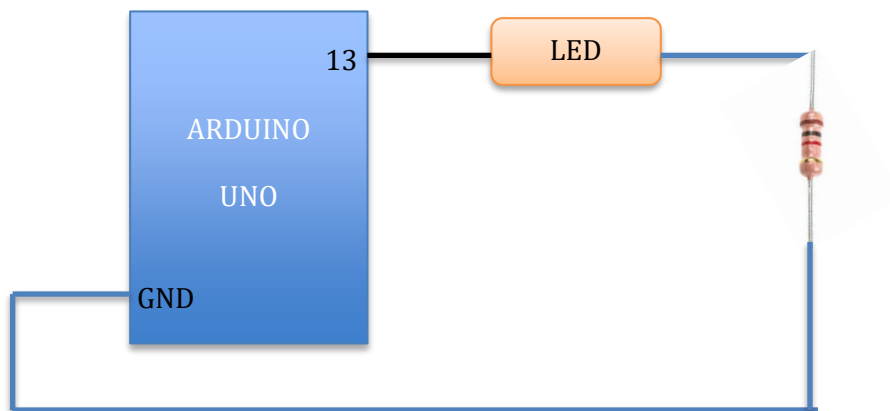
Objective:

To build a simple circuit and program an Arduino to blink an LED on and off at a regular interval using the Arduino IDE and monitor its status via the Serial Monitor.

Components Required:

S.No	Component	Quantity
1	Arduino UNO	1
2	LED	1
3	220 Ω Resistor	1
4	Breadboard	1
5	Jumper Wires	As needed
6	USB Cable (Type B)	1
7	Arduino IDE	Installed on PC

Circuit Connections:



- Connect the anode (longer leg) of the LED to digital pin 13 of the Arduino via a 220 Ω resistor.
- Connect the cathode (shorter leg) of the LED to GND on the Arduino.

Arduino Code:

```
void setup() {
  Serial.begin(9600);    // Start Serial communication
  pinMode(13, OUTPUT);   // Set digital pin 13 as output
}
```

```
Serial.println("Starting LED Blink Program...");  
}
```

```
void loop() {  
  digitalWrite(13, HIGH);  // Turn the LED ON  
  Serial.println("LED ON");  
  delay(1000);             // Wait for 1 second  
  
  digitalWrite(13, LOW);   // Turn the LED OFF  
  Serial.println("LED OFF");  
  delay(1000);             // Wait for 1 second  
}
```

Procedure:

1. Connect the components as per the circuit diagram.
2. Open the Arduino IDE on your computer.
3. Go to Tools > Board and select Arduino UNO (or your specific board).
4. Go to Tools > Port and select the correct COM port.
5. Copy and paste the above code into the editor.
6. Click the Upload button.
7. After uploading, open the Serial Monitor and set the baud rate to 9600.
8. Observe the LED blinking and the status messages printed in the Serial Monitor.

Output Observation:

- The LED connected to pin 13 turns ON for 1 second and OFF for 1 second, repeating continuously.
- The Serial Monitor displays:
Starting LED Blink Program...
LED ON
LED OFF
LED ON
LED OFF
...

Result:

Successfully built and demonstrated an Arduino LED blinking circuit using the Arduino IDE, and verified the output using Serial Monitor.

Ex7: Blink LED using ESP8266

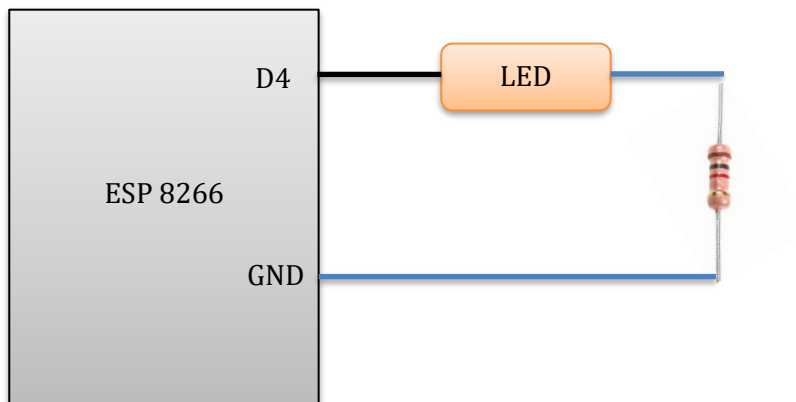
Objective:

To blink an LED using the ESP8266 (e.g., NodeMCU) microcontroller and understand basic GPIO programming along with serial output monitoring.

Components Required:

S.No	Component	Quantity
1	ESP8266 Dev Board (NodeMCU)	1
2	LED	1
3	220 Ω Resistor	1
4	Breadboard	1
5	Jumper Wires	As required

Circuit Connections:



- Connect the anode (long leg) of the LED to GPIO2 (D4) of the ESP8266 via a 220 Ω resistor.
- Connect the cathode (short leg) of the LED to GND.

Note: On the NodeMCU board, GPIO2 corresponds to the D4 pin.

Arduino Code:

```
void setup() {  
  Serial.begin(115200);    // Start Serial communication  
  pinMode(2, OUTPUT);      // Set GPIO2 (D4) as output  
  digitalWrite(2, LOW);    // Initialize LED OFF  
  Serial.println("Blinking LED on GPIO2 (D4)");  
}  
  
void loop() {  
  digitalWrite(2, HIGH);   // Turn LED ON  
  Serial.println("LED ON");  
  delay(1000);             // Wait for 1 second  
  
  digitalWrite(2, LOW);    // Turn LED OFF  
  Serial.println("LED OFF");  
  delay(1000);             // Wait for 1 second  
}
```

Procedure:

1. Assemble the circuit on a breadboard as described in the Circuit Connections section.
2. Connect the ESP8266 to your computer via USB.
3. Open the Arduino IDE.
4. Go to Tools > Board and select NodeMCU 1.0 (ESP-12E Module) or your ESP8266 board.
5. Select the correct COM port from Tools > Port.
6. Paste the above code into the Arduino IDE editor.
7. Click on the Upload button.
8. Open the Serial Monitor and set the baud rate to 115200.
9. Observe the LED blinking every second and the LED state printed to the Serial Monitor.

Output Observation:

- The LED blinks ON and OFF every 1 second.
- The Serial Monitor shows:

Blinking LED on GPIO2 (D4)
LED ON

Date:_____

LED OFF
LED ON
LED OFF
...

Result:

Successfully blinked an LED using the ESP8266. • Verified functionality using Serial Monitor messages. This experiment demonstrated the use of a digital GPIO pin on the ESP8266 to control an external LED. The Serial Monitor is helpful for debugging and confirming output in real time.

Ex 8: LDR Interfacing with ESP8266

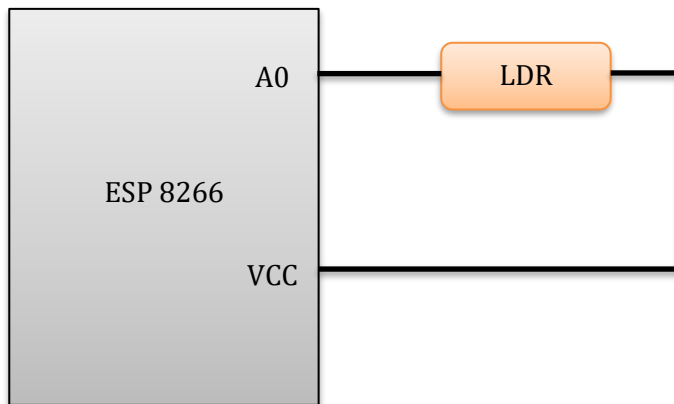
Objective:

To interface a Light Dependent Resistor (LDR) with ESP8266 (NodeMCU) and measure light intensity using analog input, displaying the result via Serial Monitor.

Components Required:

S.No	Component	Quantity
1	ESP8266 (NodeMCU)	1
2	LDR (Photoresistor)	1
3	10k Ω Resistor	1
4	Breadboard	1
5	Jumper Wires	As required
6	Arduino IDE	Installed

Circuit Connections:



- Connect one end of the LDR to 3.3V.
- Connect the other end of the LDR to A0 (analog input) and to one end of a 10k Ω resistor.
- Connect the other end of the 10k Ω resistor to GND.

This forms a voltage divider. The analog voltage at A0 varies with light intensity.

Arduino Code:

```
void setup() {  
  Serial.begin(115200); // Start Serial communication  
  Serial.println("LDR Interfacing with ESP8266 Started");  
}  
  
void loop() {  
  int ldrValue = analogRead(A0); // Read analog value from LDR  
  Serial.print("LDR Value: ");  
  Serial.println(ldrValue);      // Print to Serial Monitor  
  
  delay(1000); // Delay 1 second  
}
```

Procedure:

1. Assemble the voltage divider circuit with LDR and 10k Ω resistor on the breadboard.
2. Connect ESP8266 to the computer using a USB cable.
3. Open Arduino IDE.
4. Go to Tools > Board and select 'NodeMCU 1.0 (ESP-12E Module)'.
5. Select the correct COM port under Tools > Port.
6. Paste the provided code into the Arduino IDE editor.
7. Upload the code to the ESP8266 board.
8. Open Serial Monitor and set baud rate to 115200.
9. Observe the LDR values changing as light intensity changes.

Output Observation:

- When light intensity increases, the LDR resistance decreases, increasing the analog voltage at A0.
- The Serial Monitor displays a higher value in brighter light and lower value in darkness.

Result:

Successfully interfaced an LDR with ESP8266 and monitored light intensity using the analog input and Serial Monitor. This experiment demonstrates how to use a voltage divider circuit with an LDR to sense light intensity on the ESP8266 using analog input.

Ex 9: Object Detection using IR Sensor and Servo Motor Control

Aim:

To detect the presence of an object using an IR sensor and control a Servo Motor to rotate accordingly.

Apparatus Required:

Component	Quantity
Arduino Uno/Nano	1
IR Sensor Module	1
Servo Motor (SG90/other)	1
Jumper Wires	As required
Breadboard (optional)	1
USB Cable	1
Computer with Arduino IDE	1

Theory:

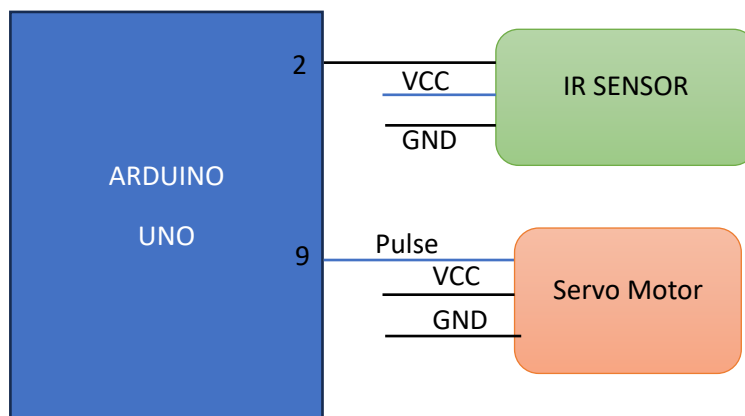
IR Sensor:

- An IR sensor detects objects based on infrared light reflection.
- Output:
 - LOW (0) – Object detected (obstacle present)
 - HIGH (1) – No object detected

Servo Motor:

- A servo motor rotates to a specified angle between 0° and 180°.
- Controlled using PWM (Pulse Width Modulation).
- Used in robotics, automation, and control systems.

Circuit Diagram:



IR Sensor OUT Pin → Arduino Digital Pin 2

IR Sensor VCC → 5V

IR Sensor GND → GND

Servo Motor Signal → Arduino Digital Pin 9

Servo Motor VCC → 5V

Servo Motor GND → GND

Procedure:

1. Connect the IR sensor to Arduino as per the circuit.
2. Connect the Servo motor to Arduino.
3. Open Arduino IDE and select the correct Board and COM Port.
4. Upload the following code to the Arduino:

Program Code:

```
#include <Servo.h>
```

```
Servo myServo;
```

```
int sensorPin = 2;
```

```
int sensorValue;
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  myServo.attach(9);
```

```
  pinMode(sensorPin, INPUT);
```

```
  Serial.println("IR Sensor & Servo initialized.");
```

```
}
```

```
void loop() {
```

```
  sensorValue = digitalRead(sensorPin);
```

```
  Serial.print("IR Sensor Value: ");
```

```
  Serial.println(sensorValue);
```

```
  if (sensorValue == LOW) {
```

```
    myServo.write(90);
```

```
    Serial.println("Object Detected - Servo at 90°");
```

```
  } else {
```

```
    myServo.write(0);
```

```
    Serial.println("No Object - Servo at 0°");
```

```
  }
```

```
  delay(500);
```

```
}
```

Working Explanation:

- When no object is in front of the IR sensor, the servo motor stays at 0°.
- When an object is detected, the servo rotates to 90°.
- Serial monitor displays the IR sensor status and servo position.

Observation Table:

S.No	Object Presence	IR Sensor Output	Servo Angle
1	No Object	HIGH (1)	0°
2	Object Present	LOW (0)	90°

Result:

The system successfully detects the object using the IR sensor and rotates the servo motor to indicate the detection.

Ex. 10: Bluetooth Controlled RGB LED Using Mobile App

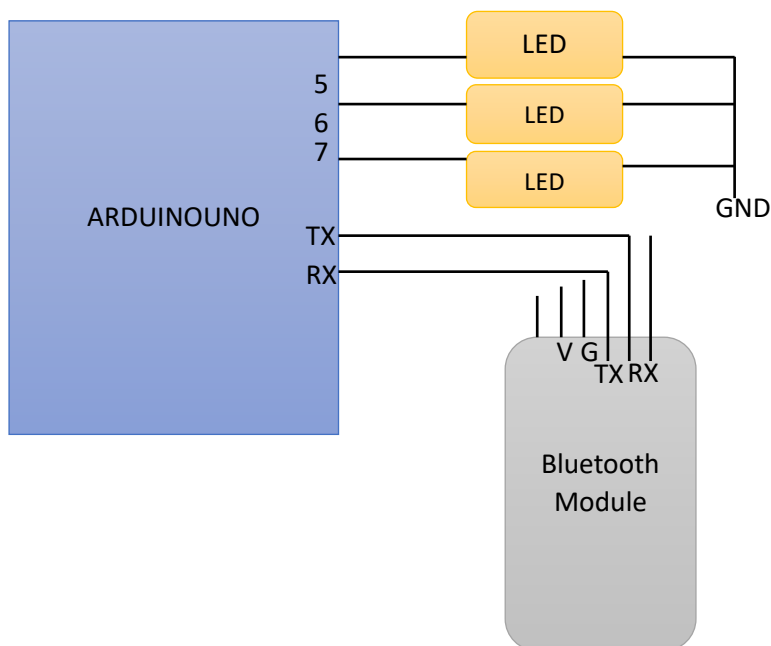
1. Aim

To design and implement a system to control an RGB LED wirelessly using Bluetooth communication and an Arduino Uno board.

2. Components Required

S. No	Component	Specification	Quantity
1	Arduino Uno	-	1
2	Bluetooth Module	HC-05 / HC-06	1
3	RGB LED	Common Cathode/Anode	1
4	Resistors	220 Ω or 330 Ω	3
5	Jumper Wires	-	As needed
6	Breadboard	-	1
7	USB Cable	For Arduino programming	1

3. Circuit Connections



1. Power Supply for RGB LED:
 - Connect the common terminal of the RGB LED:
 - If you are using a common anode RGB LED, connect its common pin to +5V on the Arduino.
 - If you are using a common cathode RGB LED, connect its common pin to GND on the Arduino.
2. Connecting LED Color Pins:
 - Connect each color leg of the RGB LED (Red, Green, and Blue) to a current-limiting resistor (220 Ω or 330 Ω recommended).

- Then connect:
 - Red LED pin → Arduino digital pin D5
 - Green LED pin → Arduino digital pin D6
 - Blue LED pin → Arduino digital pin D7
- 3. Bluetooth Module Connections:
 - Bluetooth TX pin → Connect to Arduino digital pin D2 (which acts as SoftwareSerial RX).
 - Bluetooth RX pin → Connect to Arduino digital pin D3 (which acts as SoftwareSerial TX).
 - Also, connect the VCC and GND of the Bluetooth module to the Arduino's +5V and GND pins respectively.
- 4. Common Ground:
 - Ensure the Arduino, RGB LED, and Bluetooth module share a common ground connection to complete the circuit properly.

Working Principle

- The Bluetooth module is used to receive commands wirelessly from a smartphone or PC.
- The Arduino Uno reads the received character and switches the respective color of the RGB LED.
- The SoftwareSerial library is used to create a secondary serial port for Bluetooth, leaving the default Serial for debugging.

Procedure

1. Connect the RGB LED pins (Red, Green, Blue) to Arduino digital pins 5, 6, and 7 via resistors.
2. Connect the Bluetooth module to Arduino pins 2 (RX) and 3 (TX) using the SoftwareSerial library.
3. Upload the program to the Arduino Uno.
4. Pair the Bluetooth module with your smartphone using a Bluetooth terminal app.
5. Send commands 0–3 to control the LED colors:
 - 0 → OFF
 - 1 → Red ON
 - 2 → Green ON
 - 3 → Blue ON
6. Observe the status messages on the Arduino Serial Monitor for debugging.

Observations

Command Sent	LED Color	Serial Monitor Output
0	All OFF	RGB OFF
1	Red ON	Red ON
2	Green ON	Green ON
3	Blue ON	Blue ON
Others	No change	Unknown Command

Result

The RGB LED was successfully controlled wirelessly through Bluetooth by sending commands from a mobile application. The system performed as expected, demonstrating wireless communication and color switching based on user input.

Ex 11: 3D Modeling of a Rocket using Tinkercad

Aim

To design a 3D model of a rocket using Tinkercad by applying basic geometric shapes and understanding alignment, grouping, and 3D workspace navigation.

Apparatus / Software Required:

S.No.	Item / Software	Specification
1	Computer / Laptop	With Internet
2	Web Browser	Chrome / Firefox
3	Tinkercad	www.tinkercad.com
4	Internet Connection	Stable

Theory / Background:

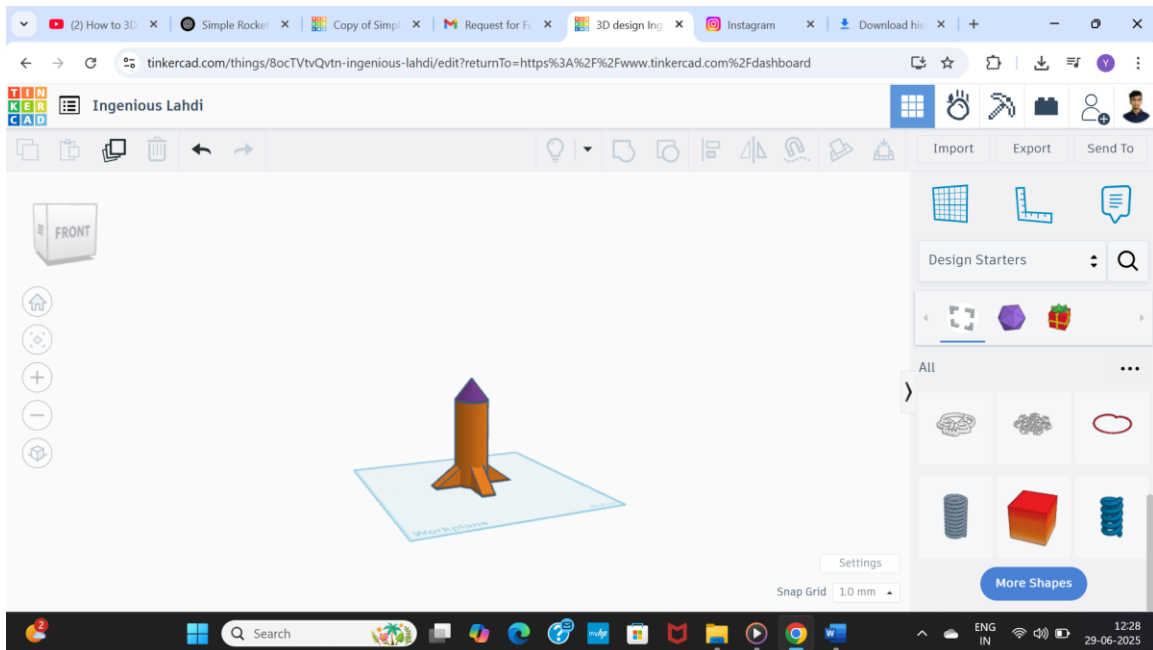
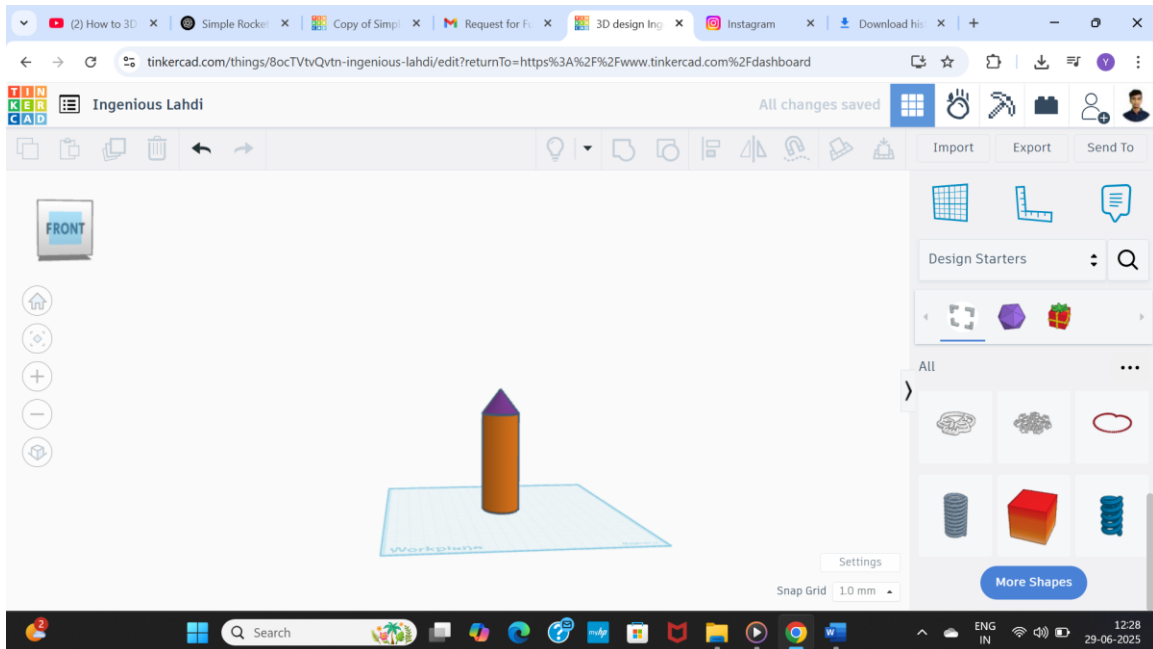
Tinkercad is a free, browser-based 3D modeling and CAD tool. It uses basic shapes such as cylinders, cones, and wedges to build complex 3D objects. In this experiment, a rocket model is created by combining these primitive shapes. The knowledge of this process helps in understanding the fundamentals of digital modeling and additive manufacturing (3D printing).

Procedure / Steps:

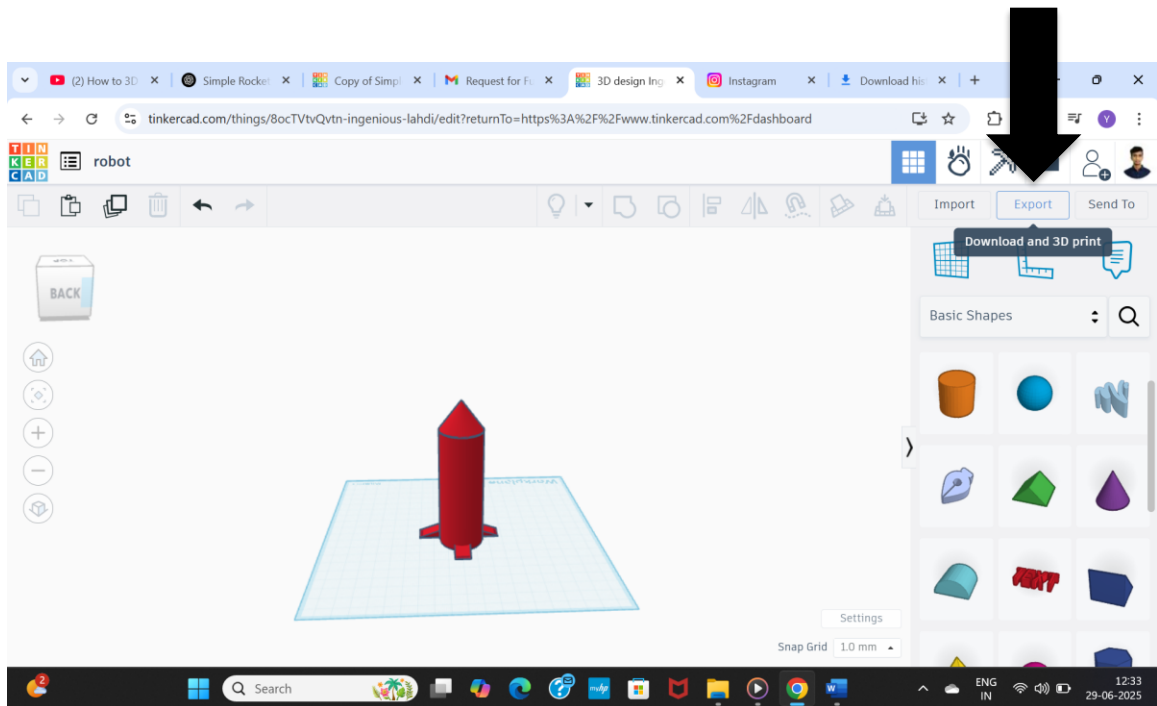
1. Open <https://www.tinkercad.com> and create a new design.
2. Drag a Cylinder to the workplane and resize it to form the rocket body (e.g., height 50 mm, diameter 15 mm).
3. Drag a Cone and place it on top of the cylinder. Align both shapes centrally using the Align Tool.
4. Create a Fin using the Wedge shape. Resize it and position it at the base of the rocket.
5. Duplicate the fin and place 3 or 4 fins equally around the base using the Rotate Tool.
6. Add a smaller Cylinder to the bottom center as the nozzle.
7. Change the colors of different parts (optional).
8. Select all objects and Group them to form a single rocket model.
9. View the model from different angles and make necessary adjustments.
10. Save your project and export it as an STL file (if needed for 3D printing).

Date: _____

5. Diagram / Screenshot:



Date: _____



Output:

Successfully created a 3D model of a rocket using basic shapes and tools in Tinkercad.

Result:

The rocket was designed successfully using Tinkercad with proper alignment, grouping, and shape manipulation.

Reference: https://www.tinkercad.com/things/8ocTVtvQvtn-obot?sharecode=knz3xtPNZg-zkDZ6ySTRDpeoGQg_TQoz7126Uck_qdg

Ex 12: Build a live soil moisture monitoring project, and monitor soil moisture levels of a remote plan in your computer dashboard

1. Aim

To measure soil moisture levels using an analog soil moisture sensor and upload the data to the Thingzmate IoT platform via Wi-Fi using an ESP8266 microcontroller.

2. Components Required

S. No	Component	Specification	Quantity
1	ESP8266 MCU	ESP-12E or similar	1
2	Soil Moisture Sensor	Analog output sensor	1
3	Jumper Wires	-	As needed
4	Breadboard	-	1
5	USB Cable	For programming the ESP	1

3. Circuit Connections

- **Soil Moisture Sensor:**
 - **VCC** → Connect to **3.3V** pin on ESP8266.
 - **GND** → Connect to **GND** pin on ESP8266.
 - **Analog Output (A0)** → Connect to **A0** pin on ESP8266.
- **ESP8266 NodeMCU:**
 - Connect to PC using micro USB cable for programming and power.

4. Working Principle

- The soil moisture sensor measures the voltage variation corresponding to soil moisture.
- The analog value is read by the ESP8266 through pin **A0**.
- The value is mapped to a percentage (%).
- The ESP8266 connects to Wi-Fi and sends the data to Thingzmate using an HTTPS POST request every 10 seconds.
- The HTTP request includes an **Authorization token** and JSON payload.

5. Program Code

```
#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <ESP8266HTTPClient.h>

// WiFi credentials
const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";

// Thingzmate details
const char* serverUrl = "https://console.thingzmate.com/api/v1/device-types/xxx/devices/soilmoisture/uplink";
const char* authToken = "Bearer YOUR_AUTH_TOKEN";

// Analog pin
#define AOUT_PIN A0

void setup() {
  Serial.begin(9600);
  delay(100);

  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi...");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println(" connected!");
}

void loop() {
  int sensorValue = analogRead(AOUT_PIN);
  float moisture = map(sensorValue, 1023, 0, 0, 100); // Adjust based on calibration

  Serial.print("Raw Sensor Value: ");
  Serial.print(sensorValue);
  Serial.print(" | Moisture: ");
  Serial.print(moisture);
  Serial.println(" %");

  // Prepare JSON payload
  String payload = "{\"data\":{\"soil_moisture\":";
  payload += moisture;
  payload += "}}";

  // Send HTTP POST
  if (WiFi.status() == WL_CONNECTED) {
```

```

HTTPClient http;
WiFiClient client;

http.begin(client, serverUrl);
http.addHeader("Content-Type", "application/json");
http.addHeader("Authorization", authToken);

int httpCode = http.POST(payload);

Serial.print("HTTP Status Code: ");
Serial.println(httpCode);

if (httpCode > 0) {
  String response = http.getString();
  Serial.println("Data sent successfully");
} else {
  Serial.println("Failed to send data.");
}
http.end();
} else {
  Serial.println("WiFi Disconnected");
}

delay(10000); // Send every 10 seconds
}

```

7. Observations

Parameter	Value Range	Unit
Analog Value	0 - 1023	-
Mapped Moisture	0 - 100	Percent
HTTP Status	200	OK

Observe Serial Monitor for:

- Wi-Fi status
- Raw and mapped sensor values
- HTTP status code (200 = success)

8. Result

The soil moisture data was successfully measured using the analog sensor and uploaded to the Thingzmate IoT platform via Wi-Fi using the ESP8266 microcontroller. The system works as an IoT-based remote soil monitoring solution.